Dataset Engineering

Data quality, diversity, quantity, acquisition, synthesis, distillation, and processing

Design the behavior you want to teach, then build data that demonstrates it. Quality, coverage, quantity, and governance all matter; scaling a flawed dataset usually scales its defects.

Source: Chip Huyen, AI Engineering: Building Applications with Foundation Models, first edition (2025). Source: book pp. 363-404 / PDF p. 387. These are original study notes, not verbatim chapter reproductions. Tool names, prices, model comparisons, and research findings reflect the book's publication period unless stated otherwise.

01 Data Curation

Source: book pp. 365-377 / PDF p. 389

Design examples for the intended learning objective

A self-supervised example is a sequence; SFT uses a prompt and response; preference tuning uses a prompt with preferred/dispreferred responses; reward-model training may use scores or comparison-derived labels. Reasoning examples can contain worked steps; tool-use examples need tool calls, observations, roles, and destinations.

Represent multi-turn clarification, failures, recovery, and follow-up when the deployed application needs them. Human interaction traces can differ from efficient model tool use, so demonstrations must teach the model's actual interface rather than simply mimic human mouse movements.

Data quality

Quality means relevant to the task, aligned with desired behavior, consistently annotated, correctly formatted, sufficiently unique, and compliant. Correctness is crucial where the task demands facts, but appropriateness also depends on audience and use.

Small curated sets can outperform large noisy sets, as illustrated by LIMA and other studies. This is not a universal sample-count rule. Resolve contradictory annotations and formatting errors; model architecture and compute cannot compensate indefinitely for poor supervision. Clear guidelines are part of the dataset, not optional paperwork.

Data coverage and mixtures

Cover tasks, topics, languages, dialects, input length, output formats, typos, conversation turns, and important rare cases. Quality and diversity complement each other; a polished set of near-identical examples is still narrow.

Pre-training, SFT, and preference phases can need different domain mixtures. The book's Llama examples show different proportions of general knowledge, math/reasoning, code, and multilingual data; those are observations, not recommended ratios for every application. Small-model experiments and learning curves can inform mixture choices. High-quality math/code annealing is an example of targeted late training, not proof that all tasks need the same data.

Data quantity and diminishing returns

Needs depend on task difficulty, method, base-model capability, and example quality. A capable base with PEFT may be attractive with little data; abundant data can make a smaller fully tuned model competitive. Treat these as hypotheses to evaluate, not hard thresholds.

Begin with a small curated pilot, often tens to hundreds of examples, and inspect held-out gains. If the pilot fails, diagnose labels, formatting, hyperparameters, model suitability, and evaluation before scaling. Plot performance versus sample count to estimate marginal value. Staged learning can use broad/raw data before task-specific data or synthetic before real data. Ossification describes limits to adapting some overtrained/small models; poor adaptation is not always fixed by adding examples.

---

02 Data Acquisition and Annotation

Source: book pp. 377-380 / PDF p. 401

Sources, annotation budgets, and the data flywheel

Sources include public datasets, licensed sources, user-approved application interactions, system traces, and explicit/implicit feedback. Application data often best matches the real task, but collection needs consent, privacy safeguards, retention rules, and access controls.

Check provenance and source rights even when a dataset has a permissive top-level license. Plan annotation cost from sample counts, expert rates, repeat labeling, adjudication, and quality review. A data flywheel is useful only when collected signals reliably improve the product, not just because more interactions are stored.

Clear annotation guidelines

Define what deserves each score, how to handle ambiguity, correctness versus helpfulness, unsafe/out-of-scope requests, and edge cases. Give concrete examples and train annotators. Compare independent labels, resolve disagreement, and revise the guide.

AI annotation needs the same clarity as human annotation. Public data hubs help discovery, but all candidate datasets need inspection for relevance, bias, quality, duplicates, and licensing.

---

03 Data Augmentation and Synthesis

Source: book pp. 380-395 / PDF p. 404

Why synthesize? Traditional techniques

Augmentation transforms existing examples; synthesis creates new examples approximating desired properties. Goals include scale, sparse-case coverage, controlled quality, privacy-aware development, testing, and distillation.

Traditional methods include templates with controlled/random fields, transformations such as paraphrases or image crops, and perturbations for robustness. Preserve labels and semantics. Simulators generate safe, repeatable trajectories for robotics, driving, or tool use; choose successful validated actions as demonstrations. Simulation coverage and realism matter as much as volume.

AI-powered data synthesis and instruction data

Generate instructions from topics/templates/seeds and produce candidate responses. Paraphrasing, translation, and controlled variations can expand coverage. Reverse instruction starts with high-quality authorized content and asks for a matching prompt, avoiding the need to fabricate the long answer itself.

Book examples include UltraChat's topic expansion, Alpaca's seed-driven instructions, and MetaMath-style mathematical variations. These demonstrate construction strategies, not permanent model rankings. Track the generator, prompt, source content, and filters. Iterative self-improvement needs independent evaluation to avoid reinforcing the generator's mistakes.

Long-context, code, preference, and back-translation workflows

Long context: generate verified QA pairs from manageable document chunks, then train with the same question and full document to teach locating evidence in longer inputs. Ensure the model/runtime supports the longer sequence; data alone does not remove architectural limits.

Code: generate problems, solutions, tests, and fixes using execution feedback; translate corrected programs across languages and test each version. The book describes millions of Llama code examples constructed this way.

Back-translation: translate text or code into an explanation and back, checking preserved meaning/function rather than identical wording. **Preference data:** generate several responses and judge them, swapping order to reduce position bias.

Data verification and limitations

Prefer functional tests and deterministic constraints where possible; otherwise combine rubric-based judges, heuristics, and expert review. Check instruction feasibility, factual support, repetition, contradictory targets, length, and topic coverage. AI-content detectors can flag distribution differences, but fooling a detector is not proof of quality.

Risks include superficial imitation, hallucinated teacher knowledge, biased samples, obscured lineage, and recursive **model collapse**: frequent patterns dominate while tails disappear. Mix trusted real data, preserve rare categories, and evaluate on independent real use. Recursion does not inevitably fail in every setting; results depend on data retention, selection, and verification.

---

04 Model Distillation

Source: book pp. 395-396 / PDF p. 419

Teacher-to-student transfer

Distillation trains a student to reproduce useful teacher behavior, through teacher outputs and sometimes richer probability signals when available. It can reduce serving cost for a specific task while retaining enough capability.

A teacher need not always have more total parameters; task expertise, model architecture, and the teaching objective matter. Not all synthetic-data training is distillation. Measure the student's actual generalization, cost, and regressions, and confirm the teacher's license permits using outputs to train another model.

---

05 Data Processing

Source: book pp. 396-404 / PDF p. 420

Inspect data before automating it

Read representative examples and inspect distributions of tasks, topics, prompt/response lengths, languages, annotation scores, and sources. Compare annotators and score distributions; resolve conflicts rather than blindly averaging.

Manual inspection often reveals defects invisible in aggregate statistics. Optimize processing pipelines for large sets, cache expensive intermediate representations, and order cheap filters before expensive verification when practical. Keep reproducible transformation records and counts.

Deduplicate data

Look for whole-document, within-document, and cross-document repetition, plus exact, lexical near-duplicates, and semantic near-duplicates. Check train/evaluation overlap separately to protect measurement integrity. Repetition can distort learning and waste compute.

All-pairs checks scale quadratically. Hash exact records; use n-grams, MinHash/LSH or related bucketing for candidates, and embeddings/similarity for semantic duplicates. Bloom filters support efficient membership checks with false positives. Reduced dimensions can speed comparisons but lose signal. Review thresholds to avoid erasing legitimate recurring patterns or minority examples.

Clean, filter, and format

Remove irrelevant markup and corrupted data while preserving structures the task needs, such as code or meaningful tables. Filter unsafe, private, unusable, and poorly aligned examples under explicit rules; importance sampling and active learning can prioritize valuable cases.

Match the target model's chat template, role boundaries, delimiters, and loss masks. Fine-tuning can shorten inference instructions, but inference format must still match training. Validate examples after formatting and prevent leakage from evaluation data. Keep dataset versions, lineage, and reproducible train/validation splits.

Revision Check

1. Which behaviors and rare cases must appear in your dataset?
2. How would you decide whether more examples are worth labeling?
3. What verifies synthetic code or long-context QA?
4. Why is a human-like synthetic example not necessarily correct?
5. How will you protect evaluation data during deduplication and formatting?

Use data engineering to teach specific capabilities, not merely accumulate records. Validate the learning effect on independent examples and preserve provenance throughout.

Book-based notes, separate from the earlier transcript-based notes. Coverage is indexed by section in coverage.md. Source links open the supplied book at the relevant PDF page. Detailed examples, citations, and complete experimental evidence remain in the book.